

# Rebase the Library Fundamentals v3 TS on C++20

## Abstract

The Library Fundamentals v3 TS should be based on C++20.

## Contents

1. [Revision history](#)
2. [Proposal](#)
3. [Proposed wording](#)
  1. [Front matter updates](#)
  2. [Deletions of merged material](#)
  3. [Systematic updates of the method of specification and presentation](#)

## Revision history

- [P2081R0](#): Initial proposal, pre-Prague 2020.
- P2081R1 (this revision): Also remove `make_array` (with discussion); provide full edit instructions for *Mandates*/... changes, and use the new “enabled hash specialization” wording.

## Proposal

The working draft of the Extensions for Library Fundamentals, Version 3 Technical Specification (“LFTSv3” or just “LFTS” for short) is currently (as of [P0996R1](#)) based on the published C++ standard, i.e. C++17. It is very unlikely, and indeed not planned at this point, that we will publish the TS before the C++20 DIS has been published (which we expect to happen at the next meeting as of the time of writing, which is Prague 2020). Therefore, we have the option of basing the TS on the C++20 DIS, and we should do this to allow us to remove

facilities that have been merged into the main standard, and to take advantage of new language facilities.

If the C++20 IS is published before we complete the PDTS ballot for the LFTS, we should update the LFTS working draft to refer to the IS instead of the DIS. The acceptance of this proposal should include approval of such a future change, which would then be an editorial matter to align the wording with the design intent.

**A noteworthy removal:** We propose to remove `make_array` from the TS. The feature was added to the TS by [N4391](#) (adopted in Lenexa 2015) together with `to_array`. Later, [P0325R4](#) (adopted in Cologne 2019) added `to_array` to the C++20 working draft, but [explained](#) that because of class template argument deduction, “we no longer need `make_array`”.

**Other removals:** Uniform container erasure was moved into C++20 by [P1209R0](#) (adopted in San Diego 2018), and `source_location` was moved by [P1208R6](#) (adopted in Cologne 2019).

## Proposed wording

### Front matter updates

Modify [1.2, general.references] paragraphs 1, 2, and 3 as follows:

#### 1.2 Normative references [general.references]

1. The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- ISO/IEC ~~14882:2017~~[14882:-](#), Programming Languages — C++

[\[Note: Under preparation. Stage at the time of publication: ISO/DIS 14882:2020. — end note\]](#)

2. ISO/IEC ~~14882:2017~~[14882:-](#) is herein called the *C++ Standard*. References to clauses within the C++ Standard are written as “~~C++17~~[C++20](#) §3.2”. The library described in ISO/IEC ~~14882:2017-clauses 20-33~~[14882:- clauses 16-32](#) is herein called the *C++ Standard Library*.

3. Unless otherwise specified, the whole of the C++ Standard's Library introduction (~~C++17 §20~~C++20 §16) is included into this Technical Specification by reference.

Modify [1.3, general.namespaces] paragraphs 2:

2. Each header described in this technical specification shall import the contents of `std::experimental::fundamentals_v3` into `std::experimental` as if by

```
namespace std::experimental {  
    inline namespace fundamentals_v3 {}  
}  
namespace std::experimental::inline fundamentals_v3 {}
```

## Deletions of merged material

Delete subclause [6, container]. This includes the deletion of `make_array`, as discussed above, which does *not* appear in C++20.

### ~~6 Containers [container]~~

#### ~~6.1 Uniform container erasure [container.erasure]~~

##### ~~6.1.1 Header synopsis [container.erasure.syn]~~

~~1. For brevity, [...]~~

~~[...] are intentionally not provided. — end note~~

#### ~~6.2 Class template array [container.array]~~

##### ~~6.2.1 Header <experimental/array> synopsis [array.syn]~~

~~[...]~~

##### ~~6.2.2 Array creartion functions [container.array.creation]~~

~~[...]~~

Delete clause [11, reflection].

### ~~11 Reflection library [reflection]~~

~~[...]~~

Delete from Table 1 — C++ library headers:

```
<experimental/algorithm>
<experimental/array>
<experimental/deque>
<experimental/forward_list>
<experimental/functional>
<experimental/future>
<experimental/iterator>
<experimental/list>
<experimental/map>
<experimental/memory>
<experimental/memory_resource>
<experimental/propagate_const>
<experimental/random>
<experimental/scope>
<experimental/set>
<experimental/source_location>
<experimental/string>
<experimental/type_traits>
<experimental/unordered_map>
<experimental/unordered_set>
<experimental/utility>
<experimental/vector>
```

Delete from Table 2 — Significant features in this technical specification:

| Doc. No.         | Title                                                                 | [...]            |
|------------------|-----------------------------------------------------------------------|------------------|
| N4388            | A Proposal to Add a Const-Propagating Wrapper to the Standard Library | [...]            |
| [...]            | [...]                                                                 | [...]            |
| <del>N4273</del> | <del>Uniform Container Erasure</del>                                  | <del>[...]</del> |
| <del>N4391</del> | <del>make_array</del>                                                 | <del>[...]</del> |
| [...]            | [...]                                                                 | [...]            |
| <del>N4519</del> | <del>Source-Code Information Capture</del>                            | <del>[...]</del> |

## Systematic updates of the method of specification and presentation

Editorially apply the new compact inline namespace syntax wherever it applies.

### 3.1.1 Header `<experimental/utility>` synopsis [`utility.syn`]

```
#include <utility>

namespace std::experimental::inline namespace fundamentals_v3 {
inline namespace fundamentals_v3 {

// 3.1.2, Class erased_type
struct erased_type { };

} // namespace fundamentals_v3
} // namespace std::experimental::inline namespace fundamentals_v3
```

[Further edits not shown.]

Update section numbers, and update the “C++17-concept” names everywhere and replace “satisfies” with “meets”, e.g. “satisfies the requirements of `CopyConstructible`” becomes “meets the *Cpp17CopyConstructible* requirements”.

### ~~20.7.7~~20.10.8 `uses_allocator` [`allocator.uses`]

#### ~~20.7.7.1~~20.10.8.1 `uses_allocator` trait [`allocator.uses.trait`]

```
template <class T, class Alloc> struct uses_allocator;
```

1. *Remarks*: Automatically detects whether `T` has a nested `allocator_type` that is convertible from `Alloc`. Meets the ~~`BinaryTypeTrait`~~`Cpp17BinaryTypeTrait` requirements (~~C++17 §23.15.1~~C++20 §20.15.1). The implementation ~~shall provide~~provides a definition [...]

[Further edits not shown, though some instances will be part of the edits of the next instruction.]

Update C++17-style *Requires/Remarks* into *Mandates/Expects/Constraints*.

### 3.2.2.3 `propagate_const` constructors [`propagate_const.ctor`]

[*Note*: The following constructors are conditionally specified as `explicit`. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — *end note*]

```
template <class U> see below constexpr  
propagate_const(propagate_const<U>&& pu);
```

Constraints: `is_constructible_v<T, U>` is true.

*Remarks:* ~~This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>`.~~ The constructor is specified as explicit if and only if `!is_convertible_v<U&&, T>`.

*Effects:* Initializes `t_` as if direct-non-list-initializing an object of type `T` with the expression `std::move(pu.t_)`.

```
template <class U> see below constexpr propagate_const(U&& pu);
```

Constraints: `is_constructible_v<T, U>` is true and `decay_t<U>` is not a specialization of `propagate_const`.

*Remarks:* ~~This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` and `decay_t<U>` is not a specialization of `propagate_const`.~~ The constructor is specified as explicit if and only if `!is_convertible_v<U&&, T>`.

*Effects:* Initializes `t_` as if direct-non-list-initializing an object of type `T` with the expression `std::forward<U>(u)`.

*Editorial note:* The rvalue reference in `is_constructible` is redundant, since it is already contained in the definition of `is_constructible` (or, more concretely, in the definition of `declval`), and therefore we remove it here.

#### 3.2.2.4 `propagate_const` assignment [`propagate_const.assignment`]

[...]

*Remarks:* ~~This function shall not participate in overload resolution unless~~  
Constraints: `U` is implicitly convertible to `T`.

[...]

*Remarks:* ~~This function shall not participate in overload resolution unless~~  
Constraints: `U` is implicitly convertible to `T` and `decay_t<U>` is not a specialization of `propagate_const`.

[...]

### 3.2.2.5 propagate\_const const observers

[propagate\_const.const\_observers]

[...]

~~Remarks: This function shall not participate in overload resolution~~  
~~unless~~Constraints: T is an object pointer type or has an implicit conversion to const element\_type\*.

[...]

### 3.2.2.6 propagate\_const non-const observers

[propagate\_const.non\_const\_observers]

[...]

~~Remarks: This function shall not participate in overload resolution~~  
~~unless~~Constraints: T is an object pointer type or has an implicit conversion to const element\_type\*.

[...]

### 3.2.2.11 propagate\_const hash support [propagate\_const.hash]

```
template <class T>
    struct
    hash<experimental::fundamentals_v3::propagate_const<T>>;
```

The specialization

hash<experimental::fundamentals\_v3::propagate\_const<T>> is enabled (C++20 §20.14.18) if and only if hash<T> is enabled. For  
When enabled, for  
an object p of type propagate\_const<T>,  
hash<experimental::fundamentals\_v3::propagate\_const<T>>()(p)  
~~shall evaluate~~evaluates to the same value as hash<T>()(p.t\_).

~~Requires: The specialization hash<T> shall be well-formed and well-defined, and shall meet the requirements of class template hash.~~

### 3.2.2.12 propagate\_const comparison function objects

[propagate\_const.comparison\_function\_objects]

[...]

***Requires******Mandates***: The specialization `equal_to<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `equal_to<T>` ~~is~~ and well-defined.

[...]

***Requires******Mandates***: The specialization `not_equal_to<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `not_equal_to<T>` ~~is~~ and well-defined.

[...]

***Requires******Mandates***: The specialization `less<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `less<T>` ~~is~~ and well-defined.

[...]

***Requires******Mandates***: The specialization `greater<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `greater<T>` ~~is~~ and well-defined.

[...]

***Requires******Mandates***: The specialization `less_equal<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `less_equal<T>` ~~is~~ and well-defined.

[...]

***Requires******Mandates***: The specialization `greater_equal<T>` ~~shall be~~is well-formed.

***Preconditions***: The specialization `greater_equal<T>` ~~is~~ and well-defined.

[...]

**Class templates `scope_exit`, `scope_fail`, and `scope_success`**  
**[`scopeguard.exit`]**

[...]

```
template <class EFP>
    explicit scope-guard(EFP&& f) noexcept(
```



```
is_nothrow_constructible_v<EF, EFP> ||  
is_nothrow_constructible_v<EF, EFP&>);
```

**Requires**~~Mandates~~: The expression `f()` ~~shall be~~is well-formed.

**Preconditions**: Calling `f()` ~~shall have~~has well-defined behavior. For `scope_exit` and `scope_fail`, calling `f()` ~~shall~~does not throw an exception.

**Effects**: If `EFP` is not an lvalue reference type and `is_nothrow_constructible_v<EF, EFP>` is true, initialize `exit_function` with `std::forward<EFP>(f)`; otherwise initialize `exit_function` with `f`. For `scope_exit` and `scope_fail`, if the initialization of `exit_function` throws an exception, calls `f()`. [Note: For `scope_success`, `f()` will not be called if the initialization fails. — *end note*]

**Throws**: Any exception thrown during the initialization of `exit_function`.

~~**Remarks**: This constructor shall not participate in overload resolution unless~~ **Constraints**: `is_same_v<remove_cvref_t<EFP>, scope-guard>` is false and `is_constructible_v<EF, EFP>` is true.

```
scope-guard(scope-guard&& rhs) noexcept(see below)
```

**Requires****Preconditions**: If `EF` is an object type:

- if `is_nothrow_move_constructible_v<EF>` is true, `EF` ~~shall meet~~meets the ~~requirements of MoveConstructible (C++17 Table 23)~~Cpp17MoveConstructible (C++20 Table 26) requirements,
- otherwise `EF` ~~shall meet~~meets the ~~requirements of CopyConstructible (C++17 Table 24)~~Cpp17CopyConstructible requirements (C++20 Table 27).

[...]

~~**Remarks**: This constructor shall not participate in overload resolution unless~~ **Constraints**: `(is_nothrow_move_constructible_v<EF> || is_copy_constructible_v<EF>)` is true.

**Remarks**: The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<EF> || is_nothrow_copy_constructible_v<EF>`.

[...]

## Class template `unique_resource` [`scopeguard.uniqueres`]

[...]

### Constructors [`scopeguard.uniqueres.ctor`]

```
unique_resource()
```

*Effects:* Value-initializes `resource` and `deleter`; `execute_on_reset` is initialized with `false`.

~~*Remarks:* This constructor shall not participate in overload resolution unless~~ *Constraints:* `is_default_constructible_v<R>` && `is_default_constructible_v<D>` is true.

```
template <class RR, class DD>  
    unique_resource(RR&& r, DD&& d) noexcept(see below)
```

~~*Requires*~~ *Mandates:* The expressions `d(r)`, `d(RESOURCE)` and `deleter(RESOURCE)` are well-formed.

*Preconditions:* Calling `d(r)`, `d(RESOURCE)` or `deleter(RESOURCE)` ~~shall have~~ *has* well-defined behavior and ~~shall~~ *does* not throw an exception.

*Effects:* [...]

*Throws:* [...]

~~*Remarks:* This constructor shall not participate in overload resolution unless~~ *Constraints:* `is_constructible_v<R1, RR>` && `is_constructible_v<D, DD>` && (`is_nothrow_constructible_v<R1, RR>` || `is_constructible_v<R1, RR>`) && (`is_nothrow_constructible_v<D, DD>` || `is_constructible_v<D, DD>`) is true. [Note: The first two conditions prohibit initialization from an rvalue reference when either `R1` or `D` is a specialization of `reference_wrapper`. — end note]

*Remarks:* The expression inside `noexcept` is equivalent to [...]

```
unique_resource(unique_resource&& rhs) noexcept(see below);
```

[...]

### **Destructor [scopeguard.uniqueres.dtor]**

[...]

### **Assignment [scopeguard.uniqueres.assign]**

```
unique_resource&operator=(unique_resource&& rhs)  
noexcept(see below);
```

*Requires**Preconditions*: If `is_nothrow_move_assignable_v<R1>` is true, R1 ~~shall satisfy the MoveAssignable requirements (C++17 Table 25)~~meets the Cpp17MoveAssignable (C++20 Table 28) requirements; otherwise R1 ~~shall satisfy the CopyAssignable requirements (C++17 Table 26)~~meets the Cpp17CopyAssignable (C++20 Table 29) requirements. If `is_nothrow_move_assignable_v<D>` is true, D ~~shall satisfy the MoveAssignable requirements (C++17 Table 25)~~meets the Cpp17MoveAssignable (C++20 Table 28) requirements; otherwise D ~~shall satisfy the CopyAssignable requirements (C++17 Table 26)~~meets the Cpp17CopyAssignable (C++20 Table 29) requirements.

*Effects*: [...]

*Throws*: [...]

*Remarks*: [...]

### **Other member functions [scopeguard.uniqueres.members]**

```
void reset() noexcept;
```

*Effects*: [...]

```
template <class RR> void reset(RR&& r);
```

*Requires**Mandates*: The expression `deleter(r)` ~~shall be~~is well-formed.

Preconditions: Calling `deleter(r)` ~~shall have~~has well-defined behavior and ~~shall~~does not throw an exception.

*Effects:* [...]

~~Remarks: This function shall not participate in overload resolution unless~~Constraints: the selected assignment expression statement assigning `resource` is well-formed.

```
void release() noexcept;
```

*Effects:* Equivalent to `execute_on_reset = false`.

```
const R& get() const noexcept;
```

*Returns:* `resource`.

```
see below operator*() const noexcept;
```

*Effects:* Equivalent to: `return *get();`

~~Remarks: This function shall not participate in overload resolution unless~~Constraints: `is_pointer_v<R>` is true and `is_void_v<remove_pointer_t<R>>` is false.

Remarks: The return type is `add_lvalue_reference_t<remove_pointer_t<R>>`.

```
R operator->() const noexcept;
```

*Returns:* `get()`.

~~Remarks: This function shall not participate in overload resolution unless~~Constraints: `is_pointer_v<R>` is true.

```
const D& get_deleter() const noexcept;
```

*Returns:* `deleter`.

## unique\_resource creation [scopeguard.uniqueres.create]

```
template <class R, class D, class S=decay_t<R>>
    unique_resource<decay_t<R>, decay_t<D>>
make_unique_resource_checked(R&& resource, const S& invalid,
    D&& d)
    noexcept(is_nothrow_constructible_v<decay_t<R>, R> &&
    is_nothrow_constructible_v<decay_t<D>, D>);
```

**RequiresMandates:** The expression `(resource == invalid ? true : false)` ~~shall be~~is well-formed.

**Preconditions:** Evaluation of the expression `(resource == invalid ? true : false)` ~~shall have~~has well-defined behavior and ~~shall~~does not throw an exception.

*Effects:* [...]

[...]

## Class template function [func.wrap.func]

[...]

### function construct/copy/destroy [func.wrap.func.con]

[...]

```
template<class F> function& operator=(F&& f);
```

*Effects:* `function(allocator_arg, ALLOCATOR_OF(*this), std::forward<F>(f)).swap(*this);`

*Returns:* `*this`.

~~**Remarks:** This assignment operator shall not participate in overload resolution unless~~**Constraints:** `declval<decay_t<F>&>()` is ~~Callable (C++17 §23.14.13.2)~~Lvalue-Callable (C++20 §20.14.16.2) for argument types `ArgTypes...` and return type `R`.

[...]

## function modifiers [func.wrap.func.mod]

```
void swap(function& other);
```

*Requires**Preconditions*: `*this->get_memory_resource() == *other.get_memory_resource()`.

*Effects*: Interchanges the targets of `*this` and `other`.

*Remarks*: The allocators of `*this` and `other` are not interchanged.

*Editorial note*: The change from “Callable” to “Lvalue-Callable” was part of the transition from C++14 to C++17 and was previously missed by [P1210R0](#).

## Non-owning pointers [memory.observer.ptr]

[...]

### observer\_ptr constructors [memory.observer.ptr.ctor]

[...]

```
template <class W2> constexpr observer_ptr(observer_ptr<W2>
other) noexcept;
```

*Postconditions*: `get() == other.get()`.

~~*Remarks*: This constructor shall not participate in overload resolution unless~~*Constraints*: `W2*` is convertible to `W*`.

### observer\_ptr observers [memory.observer.ptr.obs]

[...]

```
constexpr reference operator*() const;
```

*Requires**Preconditions*: `get() != nullptr` is true.

*Returns*: `*get()`.

*Throws*: Nothing.

[...]

### observer\_ptr hash support [memory.observer.ptr.hash]

```
template <class T> struct
hash<experimental::observer_ptr<T>>;
```

The ~~template specialization shall meet the requirements of class template hash (C++17 §23.14.15)~~ specialization is enabled (C++20 §20.14.18). For an object `p` of type `observer_ptr<T>`, `hash<observer_ptr<T>>()(p)` ~~shall evaluate~~ evaluates to the same value as `hash<T*>()(p.get())`.

## Type-erased allocator [memory.type.erased.allocator]

[...]

Additionally, class `C` ~~shall meet~~ meets the following requirements:

- `C::allocator_type` ~~shall be identical to~~ denotes `std::experimental::erased_type`.
- `X.get_memory_resource()` returns `rp_ptr`.

## Header <experimental/memory\_resource> synopsis [memory.resource.syn]

[...]

## Alias template `resource_adaptor` [memory.resource.adaptor]

### `resource_adaptor` [memory.resource.adaptor.overview]

[...] In addition to the ~~Allocator requirements (C++17 §20.5.3.5)~~ Cpp17AllocatorRequirements (C++20 §16.5.3.5), the parameter to `resource_adaptor` shall meet the following additional requirements: [...]

[...]

### `resource_adaptor_imp` member functions [memory.resource.adaptor.mem]

[...]

```
void do_deallocate(void* p, size_t bytes, size_t alignment);
```

**Requires/Preconditions:** `p` was previously allocated using `A.allocate`, where `A == m_alloc`, and not subsequently deallocated.

*Effects:* [...]

[...]

## Algorithms library [algorithms]

[...]

### Shuffle [alg.random.shuffle]

[...]

~~*Requires:* RandomAccessIterator shall satisfy the requirements of ValueSwappable (C++17 §20.5.3.2)~~  
*Preconditions:* RandomAccessIterator meets the Cpp17ValueSwappable requirements (C++20 §16.5.3.2).

*Complexity:* [...]

*Remarks:* To the extent that the implementation of this function makes use of random numbers, the per-thread engine (10.1.2.1) ~~shall serve~~serves as the implementation's source of randomness.

## Numerics library [numeric]

[...]

### Utilities [rand.util]

#### Function template **randint** [rand.util.randint]

A separate *per-thread engine* of type `default_random_engine` (C++17 §29.6.5C++20 §26.6.5), initialized to an unpredictable state, shall be maintained for each thread.

```
template<class IntType> IntType randint(IntType a, IntType b);
```

~~*Requires*~~*Preconditions:*  $a \leq b$ .

~~*Remarks:* If the template argument does not meet the requirements for IntType, the program is ill-formed (C++17 §29.6.1.1)~~  
*Mandates:* The template argument meets the requirements for a template parameter named IntType in C++20 §26.6.2.1.



*Returns:* A random integer  $i$ ,  $a \leq i \leq b$ , produced from a thread-local instance of `uniform_int_distribution<IntType>` (~~C++17~~ ~~§29.6.8.2.1~~C++20 §26.6.8.2.1) invoked with the per-thread engine.

[...]

*Editorial instruction:* The specification elements should be reordered editorially according to the order provided by [C++20, structure.specifications].